Attack Surface Map

Trust Boundaries

Where does untrusted data enter your system? Draw or describe the boundary between what you control and what you do not.

Boundary	What crosses it	Risk
Client → Router	User submitted JSON payloads	Sensitive data exposure, malformed input
Router → Server	Registration, login, and message requests	Unauthorized access, credential exposure
Server → Router	Forwarded messages and routing information	Sensitive data exposure, message misdelivery
Router → Recipient Client	Forwarded messages and sender information	Sender impersonation, message tampering

Security-Sensitive Operations

List the key operations that enforce security properties.

Operation	Property	Location in code
User registration and password storage	Accounts	`full_server.py:_register()`
Password verification on login	Authentication	`full_server.py:_login()`
2FA verification	Authentication	`full_server.py:_verify_2fa()`
Session initiation	Session Management	`full_server.py:_initiate_session()`
Message forwarding	Message Security	`full_server.py:_send_message()`

Threat Scenarios

Scope notes (from the design doc): `Payload.route` and router addresses are treated as fully secure and out of scope for tampering. Payloads on the wire are expected to be encrypted and signed, but this is not yet implemented, so those tests are expected to fail on the current build and are recorded as findings. An attacker may fully intercept or drop a payload in transit. Every field in `Payload.data` is attacker-controlled, and the server must not rely on the desktop UI for validation.

Test harness: a Python script using `requests` that POSTs hand-built `Payload` JSON directly to the server's `/payload` endpoint and to a client's `/receive` endpoint, bypassing the desktop UI. Two registered test accounts (`alice`, `bob`) and one unregistered attacker identity (`mallory`) are used throughout.

#	Property	Attack Scenario	How We Will Test It	Expected secure result
1	Accounts	Brute-force / credential stuffing on `login`	Script 1,000 rapid `login` requests for `alice` with wrong passwords, then one with the correct password; observe throttling, lockout, or delay	Rate limiting or lockout activates; correct password is also refused during lockout
2	Accounts	Duplicate or malformed registration	Send `register` for existing and edge-case usernames/passwords; check whether the	Existing account

		(existing username, case variants such as `Alice`, whitespace, empty or oversized username, weak password)	existing account is overwritten	unchanged; invalid input rejected with an error `status_code`
3	Password storage	Plaintext or weakly hashed passwords at rest	Register a user, then inspect the server's storage (in-memory structures, any files or database) for the password string; check whether the hash is salted and slow (bcrypt/argon2/scrypt)	No plaintext; salted, slow hash
4	Authentication (2FA)	Brute-force or bypass of the verification code	Script all 1,000,000 6-digit-style guesses (or a capped sample) against `2fa`; also send `initiate_session` and `message` after `login` without ever sending `2fa`	Code attempts limited; codes expire; session features refused until 2FA succeeds
5	Authentication (2FA)	Reuse or leak of the 2FA code	Reuse a valid code a second time; search server stdout and logs for the code value	Code is single-use, short-lived, and never logged
6	Session management	Message sent with no session or no login (known concern 1)	As `mallory` (never logged in), send a `message` payload with `username: alice` and `target: bob`	Rejected with an error; nothing delivered to `bob`
7	Session management	Impersonation via the `username` field	Log in as `alice`, then send `message` with `username: bob` (and `initiate_session` / `fetch_contacts` likewise); the claimed username should not be trusted	Identity is bound to the authenticated session, not the request field; mismatch rejected
8	Session management	Session fixation, expiry, and logout	Reuse an old session after the intended expiry, after re-login, and after the user's router/client address changes; send `message` without `initiate_session` to a contact	Stale or never-initiated sessions are rejected
9	Authorization	Messaging a target the sender has no session with, or fetching another user's contacts	As `alice`, `message` a `target` she has not initiated a session with; as `mallory`, call `fetch_contacts` for `alice`	Delivery and contact data refused unless authorized
10	Message confidentiality	Plaintext messages readable by a passive on-path observer	Capture the raw HTTP body at a router (or its logs) while `alice` messages `bob`; grep for the message text	Only ciphertext visible (expected to FAIL today: encryption not implemented; record as a finding)
11	Message integrity and authenticity	Tampering or forging a message in transit, or forging the `sender` shown to the recipient	POST a hand-built payload straight to `bob`'s `/receive` with `sender: alice`; modify a captured payload's `message` and replay it onward	Recipient verifies the signature and rejects unsigned or altered messages (expected to FAIL today)
12	Replay protection	Replaying a captured valid payload	Capture a legitimate `message` or `login` payload and re-send it N times, including after delay	Duplicates detected via nonce/timestamp; replays rejected (expected to FAIL today)
13	Message delivery	Interception and dropping (allowed by the threat model)	Drop a forwarded payload at the router and observe client behavior	Sender is not told "delivered" falsely; no crash or silent loss (acknowledgement or error surfaced)
14	Input validation	Malformed or hostile JSON	Send to `/payload`: invalid JSON, missing `data`/`route`, `data` that is not a JSON string, unknown or missing `type`, duplicate keys, wrong types (int/list/null in `username`), 10 MB+ bodies, deeply nested JSON, unicode and control characters, wrong `Content-`	Server returns an error and keeps running; no stack trace to the caller; size

			`Length`; send to other paths (not `/payload`)	limits enforced
15	Injection / output handling	Hostile strings that reach the client UI or storage (e.g. format strings, very long messages, path or SQL-like characters in `username`)	Register and message with such values; check the receiving desktop client display and the persistent store	Treated purely as data; no crash, truncation bug, or query manipulation
16	Key protection (known concern 2)	Private key exposure at rest or in memory	Search the project directory and client storage for key files; check file permissions and whether keys are stored unencrypted; check whether a second local user or the "local operator" can read them	Private keys encrypted at rest or access-restricted
17	Logging / data exposure (known concern 3)	Secrets in debug logs	Run a full register, login, 2FA, and message flow; search all stdout, stderr, and log files for passwords, 2FA codes, session tokens, key material, and plaintext messages (routers log payload `data`)	No secrets or plaintext content in any log
18	Availability	Resource exhaustion by one client	Open many concurrent connections and send slow or oversized requests (the node uses a threaded HTTP server); flood `register`	Rate limits and timeouts apply; legitimate requests still served
19	Information disclosure	User enumeration and verbose errors	Compare error responses and timing for existing vs non-existing usernames on `login` and `register`	Identical responses and timing; no internal details

Each scenario will be logged as: ID, steps, observed result, pass/fail, severity, and a proposed fix. Scenarios 10-12 depend on the unimplemented encryption/signing layer, so they will be re-run once it is merged.

AI Red-Team Methodology

Tool

Claude (Anthropic) will be the primary red-team tool, since it can read the full source and design doc in context. A second model (e.g. ChatGPT or Gemini) will be run on the same prompts as a cross-check, so we can see which findings are model-specific and which recur.

Prompting strategy

The strategy is run in three passes, each with a different input to the AI:

1. **Black-box, design-level.** Provide only the design doc and the attack surface map (no code). Ask: "You are a penetration tester. Given this architecture, trust boundaries, and threat model (route and router address are trusted; payloads can be intercepted, dropped, or forged), list 10 concrete attacks per security property: accounts, authentication/2FA, session management, authorization, message confidentiality, integrity, replay, input validation, logging, and availability. For each, give the attacker's capabilities, exact steps, and what a secure implementation would do."
2. **White-box, per function.** Paste each security-sensitive function (`_register`, `_login`, `_verify_2fa`, `_initiate_session`, `_send_message`, plus `Payload.deserialize` and the router/client receive handlers) one at a time. Ask: "Find vulnerabilities in this code. For each, give the line, the exploit payload, severity, and a fix. Do not report style issues."
3. **Exploit generation and adversarial follow-up.** For each credible finding, ask the AI to write a minimal proof-of-concept payload or script against our local setup. Then ask: "What did you miss? Which attacks combine two of these weaknesses (e.g. missing session check plus username spoofing plus no signing)?" Finally, feed back our known concerns and ask for what is *not* on the list, to counter anchoring.

We will use the same threat-model preamble in every prompt so the AI does not waste findings on out-of-scope items (such as tampering with `route`).

Triage and prioritization

AI output is treated as a list of hypotheses, not findings. Each suggestion goes through:

1. **Scope filter.** Discard anything that relies on modifying `route` or router addresses, or on an attacker with local operator access, unless the design doc says it is in scope.
2. **Deduplicate and map.** Merge duplicates and map each item to a property and to a row in the Threat Scenarios table (adding new rows for anything novel).
3. **Verify.** Reproduce each with a script against the running app. Classify as *confirmed*, *not reproducible* (likely a hallucination or already mitigated), or *theoretical* (plausible but untested). Only confirmed items are reported as vulnerabilities.
4. **Score.** Rank confirmed findings by Impact x Exploitability (High/Medium/Low each), plus whether it requires authentication and whether it breaks a core property (account takeover, impersonation, and message disclosure rank highest).
5. **Prioritize fixes.** Fix order is: (1) missing authentication or session checks and identity spoofing, (2) secrets in logs and key storage, (3) encryption, signing, and replay protection, (4) rate limiting and input validation, (5) low-severity hardening.
6. **Record AI quality.** For each model, note true positives, false positives, and what it missed that manual testing found, so we can report how useful the AI red-team was.

Known Concerns

Name aspects of your current implementation you already suspect are weak. This is graded for honesty, not penalized.

1. In the current implementation, session authentication is not checked when sending messages.
2. We still need to work on providing more secure ways to protect users' private keys to prevent unauthorized access.
3. Current debug logging might leak sensitive data. These sensitive debug logs need to be removed for the final version.